



Proyectos de programación

Descripción de las clases

“Extractor de prototipus de comportament o perfils”

Arnau Pons Cardet (arnau.pons.cardet@estudiantat.upc.edu)

Felipe Núñez Gómez (felipe.arturo.nunez@estudiantat.upc.edu)

Yeray Llagostera Garcia (yeray.llagostera@estudiantat.upc.edu)

Kiara Kristin Rodriguez Uribe (kiara.kristin.rodriiguez@estudiantat.upc.edu)

Víctor Huertes Montes (victor.huertes@estudiantat.upc.edu)

Grup 32.1

Versió 1.0

Ignasi Gómez-Sebastià

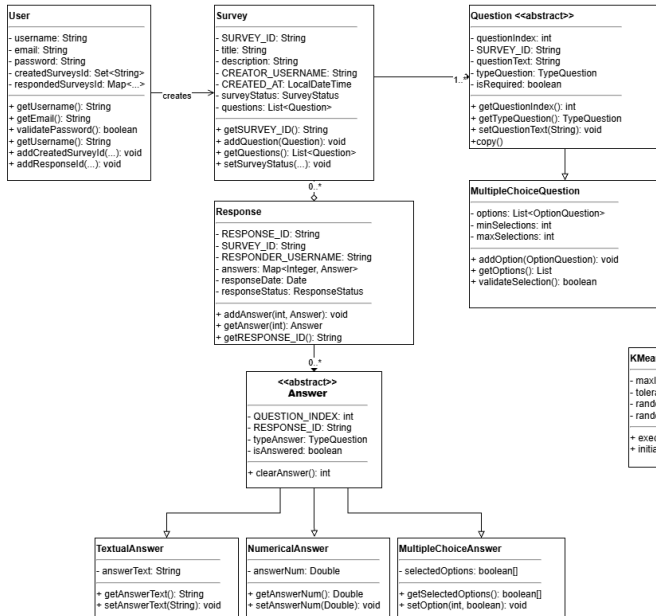
2025/2026 Q1

ÍNDICE:

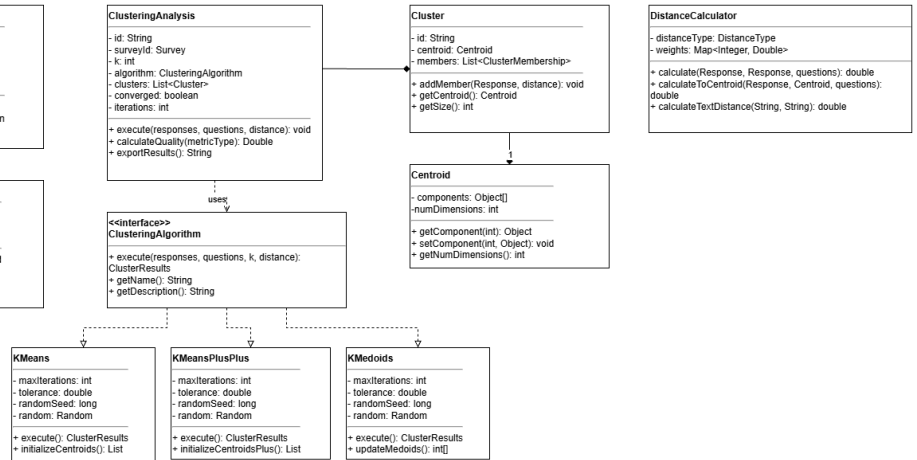
1. Diagrama UML de clases.....	2
1.1. Diseño del diagrama de clases.....	3
1.1.1. Arquitectura general.....	3
1.1.2. Capa de Modelo de Dominio.....	3
1.1.3. Capa de Clustering.....	4
1.1.4. Capa de Controladores.....	4
1.1.5. Capa de Servicios.....	4
1.2. Anotaciones y Comentarios del UML.....	5
1.3. Descripción de las clases.....	6

1. Diagrama UML de clases

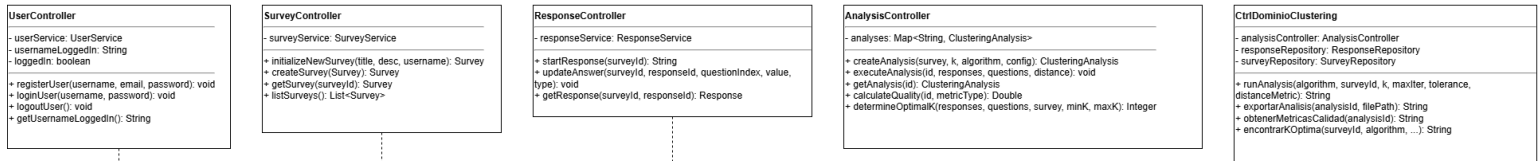
DOMAIN MODEL



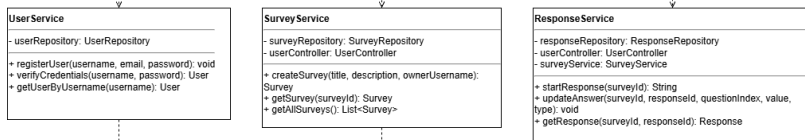
CLUSTERING DOMAIN



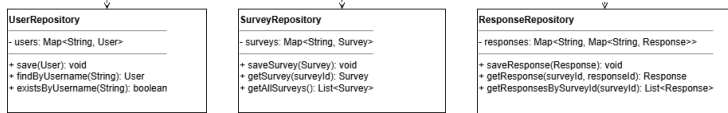
CONTROLLERS



SERVICES



REPOSITORIES



1.1. Diseño del diagrama de clases

El diagrama UML de nuestro proyecto se estructura en cuatro capas arquitectónicas principales que siguen el patrón de diseño Modelo-Vista-Controlador (MVC) extendido con una capa de persistencia en memoria. Esta organización garantiza la separación de responsabilidades, facilita el mantenimiento y permite la escalabilidad del sistema.

1.1.1. Arquitectura general

El diseño del sistema se divide en las siguientes capas:

1. **Domain Model (Modelo de Dominio):** Contiene las entidades principales del negocio y sus relaciones.
2. **Clustering Domain (Dominio de Clustering):** Implementa la lógica de análisis de datos mediante algoritmos de agrupamiento.
3. **Controllers (Controladores):** Gestionan la interacción entre la capa de presentación y la lógica de negocio.
4. **Services (Servicios):** Encapsulan la lógica de negocio y las reglas de validación.
5. **Repositories (Repositorios):** Gestionan el almacenamiento de datos en memoria.

1.1.2. Capa de Modelo de Dominio

Esta capa contiene las entidades fundamentales del sistema:

- **Gestión de Usuarios y Encuestas:**
 - **User:** Representa un usuario del sistema con autenticación y gestión de encuestas creadas y respondidas.
 - **Survey:** Modela una encuesta con su metadata (título, descripción, creador, fechas) y estado (borrador, publicada, cerrada).
 - **Question:** Clase base para preguntas con soporte para tipos textuales, numéricos y de opción múltiple.
 - **MultipleChoiceQuestion:** Especialización de Question que gestiona preguntas con opciones de respuesta configurables.
 - **OptionQuestion:** Representa una opción individual en preguntas de opción múltiple.
- **Gestión de Respuestas:**
 - **Response:** Encapsula el conjunto de respuestas de un usuario a una encuesta, con control de estado (borrador, enviada).
 - **Answer (clase abstracta):** Clase base para los diferentes tipos de respuestas.
 - **TextualAnswer, NumericalAnswer, MultipleChoiceAnswer:** Especializaciones que gestionan respuestas de cada tipo.
- **Relaciones clave:**
 - Un User puede crear múltiples Survey (1:N)
 - Una Survey contiene una o más Question (1:N composición)
 - Una Survey puede recibir múltiples Response (1:N)
 - Una Response contiene múltiples Answer (1:N composición)
 - Question tiene una relación de herencia con MultipleChoiceQuestion
 - Answer tiene relaciones de herencia con sus tres especializaciones

1.1.3. Capa de Clustering

Esta capa implementa los algoritmos de análisis y agrupamiento:

- **Componentes de Clustering:**
 - **ClusteringAnalysis:** Orquesta el proceso completo de análisis, almacenando configuración, resultados y métricas.
 - **ClusteringAlgorithm (interfaz):** Define el contrato para algoritmos de clustering mediante el patrón Strategy.
 - **KMeans, KMeansPlusPlus, KMedoids:** Implementaciones concretas de algoritmos de clustering con diferentes estrategias de inicialización.
 - **Cluster:** Representa un grupo de respuestas similares con su centroide y miembros.
 - **Centroid:** Modela el centro geométrico de un cluster en el espacio n-dimensional.
 - **ClusterMembership:** Registra la pertenencia de una respuesta a un cluster con su distancia.
 - **ClusterResults:** Encapsula los resultados de un análisis (clusters, iteraciones, convergencia).
 - **DistanceCalculator:** Implementa múltiples métricas de distancia (Euclidiana, Manhattan, Levenshtein) con soporte para ponderación de preguntas.
- **Patrones de diseño aplicados:**
 - **Strategy:** Para intercambiar algoritmos de clustering dinámicamente.
 - **Factory:** En la creación de centroides a partir de respuestas.

1.1.4. Capa de Controladores

Los controladores actúan como fachadas entre la capa de presentación y la lógica de negocio:

- **UserController:** Gestiona autenticación, registro y sesiones de usuario.
- **SurveyController:** Coordina operaciones CRUD sobre encuestas.
- **QuestionController:** Maneja la creación y modificación de preguntas.
- **ResponseController:** Gestiona el flujo de respuesta a encuestas.
- **AnswerController:** Procesa respuestas individuales a preguntas.
- **AnalysisController:** Coordina la ejecución y recuperación de análisis de clustering.
- **CtrlDominioClustering:** Controlador de alto nivel que integra análisis, exportación y determinación de K óptima.

Principio aplicado: Los controladores delegan toda la lógica de negocio a los servicios, actuando únicamente como intermediarios.

1.1.5. Capa de Servicios

Los servicios encapsulan la lógica de negocio y las validaciones:

- **UserService:** Valida credenciales, gestiona registro y mantiene integridad de usuarios.

- **SurveyService:** Implementa reglas de negocio para encuestas (validación de estados, permisos de edición).
- **QuestionService:** Valida coherencia de preguntas (tipos, opciones, rangos).
- **ResponseService:** Garantiza integridad de respuestas (validación de tipos, obligatoriedad).
- **AnswerService:** Procesa respuestas individuales con validaciones específicas por tipo.

1.1.6. Capa de Repositorios

Los repositorios implementan el patrón Repository para abstraer el almacenamiento en memoria:

- **UserRepository:** Almacena usuarios con índice por username y mantiene relaciones bidireccionales (encuestas creadas, respuestas dadas).
- **SurveyRepository:** Gestiona encuestas con generación automática de IDs y acceso por usuario creador.
- **ResponseRepository:** Implementa doble indexación (surveyId → responseId → Response) para acceso eficiente.

1.2. Anotaciones y Comentarios del UML

- **Gestión de IDs:** Todos los identificadores se generan y gestionan en los repositorios para garantizar unicidad global.
- **Estado de entidades:** Survey y Response implementan máquinas de estado con transiciones válidas:
 - Survey: DRAFT → PUBLISHED → CLOSED
 - Response: DRAFT → SUBMITTED
- **Clonación profunda:** Centroid implementa Cloneable para permitir snapshots de resultados de clustering sin referencias compartidas.
- **Extensibilidad:** El diseño permite añadir nuevos tipos de preguntas/respuestas extendiendo Question/Answer sin modificar código existente.
- **Thread-safety:** El diseño actual no es thread-safe. Para entornos concurrentes se requeriría sincronización en repositorios.
- **Límites de escalabilidad:** El almacenamiento en memoria limita el volumen de datos. Para producción se requeriría persistencia en base de datos.

1.3. Descripción de las clases

1.3.1. Paquete: data

SurveyRepository: Clase que gestiona el almacenamiento y recuperación de encuestas y sus preguntas asociadas en memoria.

Atributos:

- `surveys: Map<String, Survey>` privado que asocia IDs de encuestas con objetos Survey (cada Survey contiene sus preguntas).
- `nextSurveyId:` entero privado que actúa como contador para generar IDs autoincrementales de encuestas.

Métodos:

- `SurveyRepository()`: constructor que inicializa el mapa de encuestas vacío y el contador en 0.
- `void addSurvey(Survey survey)`: añade una encuesta al repositorio.
- `void deleteSurvey(String surveyId)`: elimina una encuesta del repositorio.
- `Survey getSurvey(String surveyId)`: retorna la encuesta correspondiente al ID especificado.
- `List<Survey> getAllSurveys()`: retorna una lista con todas las encuestas almacenadas.
- `List<String> getAllSurveyIds()`: retorna una lista con todos los IDs de encuestas.
- `void updateSurvey(String surveyId, Survey updatedSurvey)`: actualiza una encuesta existente.
- `boolean existsSurvey(String surveyId)`: verifica si existe una encuesta con el ID especificado.
- `List<Survey> getSurveysByUsername(String username)`: retorna las encuestas creadas por un usuario específico.
- `String generateNextSurveyId()`: genera y retorna el siguiente ID autoincremental para una nueva encuesta.
- `int getNextSurveyIdValue()`: retorna el valor actual del contador de IDs.
- `boolean canCreateMoreSurveys()`: verifica si se pueden crear más encuestas (no se ha alcanzado el límite).
- `void addQuestion(String surveyId, Question question)`: añade una pregunta a una encuesta específica.
- `void deleteQuestion(String surveyId, int questionIndex)`: elimina una pregunta de una encuesta.
- `void updateQuestion(String surveyId, int questionIndex, Question question)`: actualiza una pregunta existente.
- `Question getQuestion(String surveyId, int questionIndex)`: retorna una pregunta específica.
- `List<Question> getAllQuestions(String surveyId)`: retorna todas las preguntas de una encuesta.
- `int getNumQuestions(String surveyId)`: retorna el número de preguntas de una encuesta.
- `void swapQuestions(String surveyId, int oldQuestionIndex, int newQuestionIndex)`: reordena las preguntas de una encuesta.
- `void deleteAllQuestions(String surveyId)`: elimina todas las preguntas de una encuesta.

ResponseRepository: Clase que gestiona el almacenamiento y recuperación de respuestas a encuestas en memoria con doble indexación (por encuesta y por respuesta).

Atributos:

- **responses:** `Map<String, Map<String, Response>>` privado que asocia IDs de encuestas con un mapa de IDs de respuestas y objetos `Response`.
- **lastResponseld:** `String` privado que almacena el último ID de respuesta generado.

Métodos:

- **ResponseRepository():** constructor que inicializa el mapa de respuestas vacío y `lastResponseld` en "0".
- **String getLastResponseld():** retorna el último ID de respuesta generado.
- **void setLastResponseld(String lastResponseld):** establece el último ID de respuesta.
- **void addSurveyEntry(String surveyId):** crea una entrada para una encuesta si no existe (llamado la primera vez que un usuario responde una encuesta).
- **void deleteSurveyEntry(String surveyId):** elimina todas las respuestas asociadas a una encuesta.
- **boolean existsSurveyEntry(String surveyId):** verifica si existe una entrada para la encuesta.
- **void addResponse(String surveyId, Response response):** añade una respuesta a una encuesta.
- **Response getResponse(String surveyId, String responseld):** retorna una respuesta específica.
- **List<Response> getAllResponses(String surveyId):** retorna todas las respuestas de una encuesta.
- **void deleteResponse(String surveyId, String responseld):** elimina una respuesta específica.
- **void updateResponse(String surveyId, Response response):** actualiza una respuesta existente.
- **boolean existsResponse(String surveyId, String responseld):** verifica si existe una respuesta.
- **List<Response> getResponsesBySurveyId(String surveyId):** retorna todas las respuestas de una encuesta.
- **void updateAnswer(String surveyId, String responseld, int answerIndex, Answer answer):** actualiza una respuesta específica dentro de un objeto `Response`.
- **Answer getAnswer(String surveyId, String responseld, int answerIndex):** retorna una respuesta específica.
- **List<Answer> getAllAnswers(String surveyId, String responseld):** retorna todas las respuestas de un objeto `Response`.

UserRepository: Clase que gestiona el almacenamiento y recuperación de usuarios y sus relaciones con encuestas creadas y respondidas.

Atributos:

users: `Map<String, User>` privado que asocia nombres de usuario con objetos `User`.

Métodos:

- **UserRepository():** constructor que inicializa el mapa de usuarios vacío.
- **void addUser(User user):** añade un usuario al repositorio.
- **void deleteUser(String username):** elimina un usuario del repositorio.

- `User getUser(String username)`: retorna el usuario correspondiente al nombre especificado (debe usarse `existsUser` antes para evitar `NullPointerException`).
- `Set<String> getAllUsernames()`: retorna un conjunto con todos los nombres de usuario.
- `void updateUser(String username, User updatedUser)`: actualiza un usuario existente.
- `boolean existsUser(String username)`: verifica si existe un usuario.
- `void addSurveyId(String username, String surveyId)`: añade un ID de encuesta creada por el usuario.
- `void deleteSurveyId(String username, String surveyId)`: elimina un ID de encuesta del usuario.
- `boolean existsSurveyId(String username, String surveyId)`: verifica si el usuario creó una encuesta.
- `Set<String> getAllSurveyIds(String username)`: retorna todos los IDs de encuestas creadas por el usuario.
- `void addRespondedSurveyIdEntry(String username, String surveyId)`: crea una entrada para encuestas respondidas (mantiene coherencia con la doble indexación).
- `void deleteRespondedSurveyIdEntry(String username, String surveyId)`: elimina una entrada de encuesta respondida.
- `void addResponseId(String username, String surveyId, String responseId)`: añade un ID de respuesta del usuario a una encuesta.
- `void deleteResponseId(String username, String surveyId, String responseId)`: elimina un ID de respuesta.
- `boolean existsResponseId(String username, String surveyId, String responseId)`: verifica si existe una respuesta específica.
- `Set<String> getAllResponseIdsFromSurvey(String username, String surveyId)`: retorna todos los IDs de respuestas del usuario para una encuesta.

1.3.2. Paquete: domain.model

Survey: Clase que representa una encuesta con sus atributos y preguntas asociadas.

Atributos:

- `SURVEY_ID`: String final privado que identifica unívocamente la encuesta.
- `title`: String privado que contiene el título de la encuesta.
- `description`: String privado con la descripción de la encuesta.
- `CREATOR_USERNAME`: String final privado que identifica al usuario creador de la encuesta.
- `CREATED_AT`: LocalDateTime final privado con la fecha de creación.
- `PUBLISHED_AT`: LocalDateTime privado con la fecha de publicación.
- `surveyStatus`: SurveyStatus privado que indica el estado de la encuesta (DRAFT, PUBLISHED, CLOSED).
- `avgRating`: double privado con la valoración promedio de la encuesta.
- `views`: entero privado con el número de respuestas/visualizaciones.
- `questions`: List<Question> privada con las preguntas de la encuesta.

Métodos:

- `Survey(String SURVEY_ID, String title, String description, String CREATOR_USERNAME)`: constructor con ID para encuestas existentes.
- `Survey(String title, String description, String CREATOR_USERNAME)`: constructor sin ID para nuevas encuestas.
- `String getSURVEY_ID()`: retorna el ID de la encuesta.
- `String getTitle()`: retorna el título.
- `String getDescription()`: retorna la descripción.
- `String getCREATOR_USERNAME()`: retorna el nombre de usuario del creador.
- `LocalDateTime getCREATED_AT()`: retorna la fecha de creación.
- `LocalDateTime getPUBLISHED_AT()`: retorna la fecha de publicación.
- `SurveyStatus getSurveyStatus()`: retorna el estado de la encuesta.
- `double getAvgRating()`: retorna la valoración promedio.
- `int getViews()`: retorna el número de visualizaciones.
- `List<Question> getQuestions()`: retorna la lista de preguntas.
- `void setTitle(String title)`: establece el título.
- `void setDescription(String description)`: establece la descripción.
- `void setPUBLISHED_AT()`: establece la fecha de publicación al momento actual.
- `void setSurveyStatus(SurveyStatus surveyStatus)`: establece el estado de la encuesta.
- `void setAvgRating(double avgRating)`: establece la valoración promedio.
- `void setViews(int views)`: establece el número de visualizaciones.
- `void addQuestion(Question question)`: añade una pregunta a la encuesta.
- `void removeQuestion(int questionIndex)`: elimina una pregunta por índice.
- `void updateQuestion(int questionIndex, Question question)`: actualiza una pregunta existente.
- `Question getQuestion(int questionIndex)`: retorna una pregunta por índice.
- `int getSize()`: retorna el número de preguntas.
- `void reorderQuestion(int oldIndex, int newIndex)`: reordena una pregunta.
- `void clearQuestions()`: elimina todas las preguntas.

Question: Clase base que representa una pregunta de encuesta.

Atributos:

- `questionIndex`: entero privado que identifica la posición de la pregunta en la encuesta.
- `SURVEY_ID`: String final privado que identifica la encuesta a la que pertenece.
- `typeQuestion`: `TypeQuestion` privado que indica el tipo de pregunta (`TEXTUAL`, `NUMERICAL`, `MULTIPLECHOICE`).
- `questionText`: String privado con el texto de la pregunta.
- `isRequired`: boolean privado que indica si la pregunta es obligatoria.

Métodos:

- `Question(int questionIndex, String SURVEY_ID)`: constructor que inicializa una pregunta con tipo `TEXTUAL` por defecto y como obligatoria.
- `int getQuestionIndex()`: retorna el índice de la pregunta.
- `String getSURVEY_ID()`: retorna el ID de la encuesta.
- `TypeQuestion getTypeQuestion()`: retorna el tipo de pregunta.
- `String getQuestionText()`: retorna el texto de la pregunta.
- `boolean isRequired()`: retorna si la pregunta es obligatoria.

- `void setQuestionIndex(int questionIndex)`: establece el índice.
- `void setTypeQuestion(TypeQuestion typeQuestion)`: establece el tipo de pregunta.
- `void setQuestionText(String questionText)`: establece el texto.
- `void setRequired(boolean required)`: establece si es obligatoria.
- `Question copy()`: crea una copia profunda de la pregunta.

MultipleChoiceQuestion: Clase que representa una pregunta de opción múltiple, extiende `Question`.

Atributos:

- `minSelections`: entero privado con el número mínimo de opciones que se pueden seleccionar.
- `maxSelections`: entero privado con el número máximo de opciones que se pueden seleccionar.
- `optionQuestions`: `List<OptionQuestion>` privada con las opciones de la pregunta.

Métodos:

- `MultipleChoiceQuestion(int questionIndex, String surveyId)`: constructor que inicializa con `minSelections` y `maxSelections` en 1.
- `int getMinSelections()`: retorna el mínimo de selecciones.
- `int getMaxSelections()`: retorna el máximo de selecciones.
- `List<OptionQuestion> getOptions()`: retorna la lista de opciones.
- `int getOptionsSize()`: retorna el número de opciones.
- `void setMinSelections(int minSelections)`: establece el mínimo de selecciones con validaciones.
- `void setMaxSelections(int maxSelections)`: establece el máximo de selecciones con validaciones.
- `boolean inRange(int index)`: verifica si un índice está dentro del rango de opciones.
- `void addOption(OptionQuestion optionQuestion)`: añade una opción.
- `void removeOption(int index)`: elimina una opción por índice.
- `void reorderOption(int oldIndex, int newIndex)`: reordena una opción.
- `void updateOption(int index, OptionQuestion optionQuestion)`: actualiza una opción.
- `OptionQuestion getOption(int index)`: retorna una opción por índice.
- `void clearOptions()`: elimina todas las opciones.
- `MultipleChoiceQuestion copy()`: crea una copia profunda de la pregunta con sus opciones.

OptionQuestion: Clase que representa una opción de respuesta en preguntas de opción múltiple.

Atributos:

- `questionIndex`: entero privado que identifica la pregunta a la que pertenece.
- `surveyId`: `String` final privado que identifica la encuesta.
- `optionText`: `String` privado con el texto de la opción.

Métodos:

- `OptionQuestion(int questionIndex, String surveyId)`: constructor que inicializa la opción.
- `int getQuestionIndex()`: retorna el índice de la pregunta.
- `String getSurveyId()`: retorna el ID de la encuesta.
- `String getOptionText()`: retorna el texto de la opción.

- `void setQuestionIndex(int questionIndex)`: establece el índice de la pregunta.
- `void setOptionText(String optionText)`: establece el texto de la opción.

Response: Clase que representa un conjunto de respuestas de un usuario a una encuesta.

Atributos:

- `RESPONSE_ID`: String final privado que identifica la respuesta.
- `SURVEY_ID`: String final privado que identifica la encuesta respondida.
- `RESPONDER_USERNAME`: String final privado que identifica al usuario que respondió.
- `responseStatus`: `ResponseStatus` privado que indica el estado (DRAFT, SUBMITTED).
- `SUBMITTED_AT`: `LocalDateTime` privado con la fecha de envío.
- `ANSWERS`: array de `Answer` final privado con las respuestas a cada pregunta.

Métodos:

- `Response(String RESPONSE_ID, String SURVEY_ID, String RESPONDER_USERNAME, List<Question> questions)`: constructor con ID que inicializa las respuestas según los tipos de preguntas.
- `Response(String SURVEY_ID, String RESPONDER_USERNAME, List<Question> questions)`: constructor sin ID para nuevas respuestas.
- `String getRESPONSE_ID()`: retorna el ID de la respuesta.
- `String getSurveyId()`: retorna el ID de la encuesta.
- `String getResponderUsername()`: retorna el nombre del usuario.
- `ResponseStatus getResponseStatus()`: retorna el estado de la respuesta.
- `LocalDateTime getSUBMITTED_AT()`: retorna la fecha de envío.
- `Answer[] getANSWERS()`: retorna el array de respuestas.
- `void setResponseStatus(ResponseStatus responseStatus)`: establece el estado.
- `void setSUBMITTED_AT(LocalDateTime SUBMITTED_AT)`: establece la fecha de envío.
- `boolean inRange(int index)`: verifica si un índice está dentro del rango de respuestas.
- `void updateAnswer(int index, Answer answer)`: actualiza una respuesta por índice.
- `Answer getAnswer(int index)`: retorna una respuesta por índice.

Answer: Clase abstracta base para los diferentes tipos de respuestas.

Atributos:

- `QUESTION_INDEX`: entero final privado que identifica la pregunta respondida.
- `RESPONSE_ID`: String final privado que identifica la respuesta a la que pertenece.
- `typeAnswer`: `TypeQuestion` final privado que indica el tipo de respuesta.
- `isAnswered`: boolean privado que indica si la pregunta ha sido respondida.

Métodos:

- `Answer(int QUESTION_INDEX, String RESPONSE_ID, TypeQuestion typeAnswer)`: constructor que inicializa con `isAnswered` en false.
- `int getQUESTION_INDEX()`: retorna el índice de la pregunta.
- `String getResponseId()`: retorna el ID de la respuesta.
- `TypeQuestion getTypeAnswer()`: retorna el tipo de respuesta.
- `boolean getIsAnswered()`: retorna si ha sido respondida.

- `void setIsAnswered(boolean isAnswered)`: establece el estado de respondida (protegido).
- `abstract void clearAnswer()`: método abstracto para limpiar la respuesta.

TextualAnswer: Clase que representa una respuesta textual, extiende Answer.

Atributos:

- `answerText`: String privado con el texto de la respuesta.

Métodos:

- `TextualAnswer(int questionIndex, String responseld)`: constructor que inicializa con tipo TEXTUAL.
- `String getAnswerText()`: retorna el texto de la respuesta.
- `void setAnswerText(String answerText)`: establece el texto y marca como respondida.
- `void clearAnswer()`: limpia el texto y marca como no respondida.

NumericalAnswer: Clase que representa una respuesta numérica, extiende Answer.

Atributos:

- `answerNum`: Double privado con el valor numérico de la respuesta.

Métodos:

- `NumericalAnswer(int questionIndex, String responseld)`: constructor que inicializa con tipo NUMERICAL.
- `Double getAnswerNum()`: retorna el valor numérico.
- `void setAnswerNum(Double answerNum)`: establece el valor y marca como respondida.
- `void clearAnswer()`: limpia el valor y marca como no respondida.

MultipleChoiceAnswer: Clase que representa una respuesta de opción múltiple, extiende Answer.

Atributos:

- `maxOptions`: entero final privado con el número máximo de opciones.
- `selectedOptions`: array de boolean final privado que indica qué opciones están seleccionadas.

Métodos:

- `MultipleChoiceAnswer(int questionIndex, String responseld, int maxOptions)`: constructor que inicializa el array de opciones.
- `int getMaxOptions()`: retorna el número máximo de opciones.
- `boolean[] getSelectedOptions()`: retorna el array de opciones seleccionadas.
- `boolean inRange(int index)`: verifica si un índice está en el rango válido.
- `boolean isOptionSelected(int index)`: verifica si una opción está seleccionada.
- `void setOption(int index, boolean option)`: establece el estado de una opción y marca como respondida.
- `void setOptions(boolean[] options)`: establece todas las opciones.
- `void clearAnswer()`: limpia todas las opciones y marca como no respondida.

User: Clase que representa un usuario del sistema.

Atributos:

- **USERNAME:** String final privado que identifica unívocamente al usuario.
- **email:** String privado con el correo electrónico.
- **password:** String privado con la contraseña.
- **createdSurveyId:** `Set<String>` final privado con los IDs de encuestas creadas.
- **respondedSurveyId:** `Map<String, Set<String>>` final privado que mapea IDs de encuestas con conjuntos de IDs de respuestas.

Métodos:

- **User(String USERNAME, String email, String password):** constructor que inicializa los conjuntos vacíos.
- **String getUsername():** retorna el nombre de usuario.
- **String getEmail():** retorna el email.
- **String getPassword():** retorna la contraseña.
- **Set<String> getCreatedSurveyId():** retorna los IDs de encuestas creadas.
- **Map<String, Set<String>> getRespondedSurveyId():** retorna el mapa de encuestas respondidas.
- **void setEmail(String newEmail):** establece un nuevo email.
- **void setPassword(String newPassword):** establece una nueva contraseña.
- **void addCreatedSurveyId(String surveyId):** añade un ID de encuesta creada.
- **void removeCreatedSurveyId(String surveyId):** elimina un ID de encuesta creada.
- **boolean hasCreatedSurveyId(String surveyId):** verifica si creó una encuesta.
- **int getTotalSurveysCreated():** retorna el número de encuestas creadas.
- **void addRespondedSurveyIdEntry(String surveyId):** crea entrada para encuesta respondida.
- **void removeRespondedSurveyIdEntry(String surveyId):** elimina entrada de encuesta respondida.
- **boolean existsRespondedSurveyIdEntry(String surveyId):** verifica si existe entrada.
- **void addResponseId(String surveyId, String responseId):** añade ID de respuesta.
- **Set<String> getResponseIdsForSurvey(String surveyId):** retorna IDs de respuestas para una encuesta.
- **void removeResponseId(String surveyId, String responseId):** elimina ID de respuesta.
- **boolean hasResponseId(String surveyId, String responseId):** verifica si existe respuesta.

1.3.3. Paquete: domain.model.enums

TypeQuestion: Enumeración que define los tipos de preguntas disponibles.

Valores:

- **MULTIPLECHOICE:** pregunta de opción múltiple.
- **TEXTUAL:** pregunta de texto libre.
- **NUMERICAL:** pregunta numérica.

SurveyStatus: Enumeración que define los estados posibles de una encuesta.

Valores:

- **DRAFT:** encuesta en borrador.
- **PUBLISHED:** encuesta publicada.
- **CLOSED:** encuesta cerrada.

ResponseStatus: Enumeración que define los estados posibles de una respuesta.

Valores:

- **DRAFT:** respuesta en borrador.
- **SUBMITTED:** respuesta enviada.

1.3.4. Paquete: domain.clustering

Centroid: Clase que representa el centro de un clúster en el espacio n-dimensional, implementa Cloneable.

Atributos:

- **components:** `List<Object>` privada que contiene los valores del centroide (puede contener Double, double[] o String).
- **numDimensions:** entero final privado con el número de dimensiones (preguntas).

Métodos:

- **Centroid(int numDimensions):** constructor que inicializa la lista de componentes con valores null.
- **int getNumDimensions():** retorna el número de dimensiones.
- **List<Object> getComponents():** retorna la lista de componentes.
- **Object getComponent(int index):** retorna el componente en un índice específico.
- **void setComponent(int index, Object value):** establece el valor de un componente.
- **Centroid clone():** crea una copia profunda del centroide (clona arrays mutables).
- **String toString():** retorna una representación en String del centroide.

Cluster: Clase que representa un clúster de datos con su centroide y miembros.

Atributos:

- **id:** String final privado con el identificador único del clúster.
- **label:** String privado con la etiqueta legible del clúster.
- **centroid:** Centroid privado que representa el centro del clúster.
- **members:** `List<ClusterMembership>` final privada con los miembros del clúster.

Métodos:

- **Cluster(String id):** constructor que inicializa el clúster con un ID y label igual al ID.
- **String getId():** retorna el ID del clúster.
- **String getLabel():** retorna la etiqueta.
- **void setLabel(String label):** establece la etiqueta.
- **Centroid getCentroid():** retorna el centroide.
- **void setCentroid(Centroid centroid):** establece el centroide.
- **void addMember(Response response, double distance):** añade un miembro al clúster con su distancia.

- `boolean removeMember(String responseld)`: elimina un miembro por ID.
- `List<ClusterMembership> getMembers()`: retorna una lista inmodificable de miembros.
- `int getSize()`: retorna el número de miembros.
- `boolean contains(String responseld)`: verifica si un Response es miembro.
- `double getAverageDistance()`: calcula la distancia promedio de todos los miembros al centroide.
- `boolean equals(Objecto)`: compara clústeres por ID.
- `int hashCode()`: retorna el hash basado en el ID.

ClusterMembership: Clase que representa la pertenencia de una respuesta a un clúster.

Atributos:

- `responseld`: String final privado con el ID del Response miembro.
- `clusterId`: String final privado con el ID del clúster.
- `distance`: double privado con la distancia al centroide.
- `assignmentDate`: Date final privado con la fecha de asignación.

Métodos:

- `ClusterMembership(Response response, Cluster cluster, double distance)`: constructor que crea el registro de membresía.
- `String getResponseld()`: retorna el ID del miembro.
- `String getClusterId()`: retorna el ID del clúster.
- `double getDistance()`: retorna la distancia al centroide.
- `void setDistance(double distance)`: actualiza la distancia.

ClusterResults: Clase que encapsula los resultados de un análisis de clustering.

Atributos:

- `clusters`: `List<Cluster>` final privada con los clústeres resultantes.
- `iterations`: entero privado con el número de iteraciones realizadas.
- `converged`: boolean privado que indica si el algoritmo convergió.

Métodos:

- `ClusterResults(List<Cluster> clusters, int iterations, boolean converged)`: constructor que inicializa los resultados.
- `List<Cluster> getClusters()`: retorna la lista de clústeres.
- `Cluster getCluster(int clusterIndex)`: retorna un clúster por índice.
- `int getNumberOfClusters()`: retorna el número de clústeres.
- `int getK()`: retorna el número de clústeres (alias de `getNumberOfClusters()`).
- `int getNumberOfResponses()`: retorna el número total de respuestas en todos los clústeres.
- `int getIterations()`: retorna el número de iteraciones.
- `boolean hasConverged()`: retorna si el algoritmo convergió.
- `Integer[] getClusterAssignments()`: retorna un array con las asignaciones de clúster.

ClusteringAlgorithm: Interfaz que define el contrato para algoritmos de clustering (patrón Strategy).

Métodos:

- `ClusterResults execute(List<Response> responses, List<Question> questions, int k, DistanceCalculator distance)`: ejecuta el algoritmo de clustering y retorna los resultados.
- `String getName()`: retorna el nombre del algoritmo.
- `String getDescription()`: retorna la descripción del algoritmo.

KMeans: Clase que implementa el algoritmo K-Means con enfoque híbrido, implementa ClusteringAlgorithm.

Atributos:

- `maxIterations`: entero privado con el número máximo de iteraciones permitidas.
- `tolerance`: double privado con el umbral de convergencia.
- `randomSeed`: long privado con la semilla aleatoria para reproducibilidad.
- `random`: Random privado para generación de números aleatorios.

Métodos:

- `KMeans(int maxIterations, double tolerance)`: constructor con parámetros personalizados.
- `KMeans()`: constructor por defecto (100 iteraciones, tolerancia 1e-4).
- `void setRandomSeed(long seed)`: establece la semilla aleatoria.
- `String getName()`: retorna "K-Means".
- `String getDescription()`: retorna descripción del algoritmo.
- `ClusterResults execute(List<Response> responses, List<Question> questions, int k, DistanceCalculator distance)`: ejecuta K-Means y retorna resultados.
- Además de métodos privados auxiliares para inicialización, asignación, actualización de centroides, convergencia, etc.

KMeansPlusPlus: Clase que implementa K-Means++ con inicialización inteligente de centroides, implementa ClusteringAlgorithm.

Atributos:

- `maxIterations`: entero privado con el número máximo de iteraciones.
- `tolerance`: double privado con el umbral de convergencia.
- `randomSeed`: long privado con la semilla aleatoria.
- `random`: Random privado para números aleatorios.

Métodos:

- `KMeansPlusPlus(int maxIterations, double tolerance)`: constructor con parámetros personalizados.
- `KMeansPlusPlus()`: constructor por defecto.
- `void setRandomSeed(long seed)`: establece la semilla aleatoria.
- `String getName()`: retorna "K-Means++".
- `String getDescription()`: retorna descripción del algoritmo.
- `ClusterResults execute(List<Response> responses, List<Question> questions, int k, DistanceCalculator distance)`: ejecuta K-Means++ y retorna resultados.

- `static Centroid createCentroidFromResponse(Response r, List<Question> questions)`: crea un centroide a partir de una respuesta.
- `static boolean isAnswered(Answer a)`: verifica si una respuesta ha sido contestada.
- `static Object getAnswerValue(Answer a, Question q)`: extrae el valor de una respuesta.
- Además de métodos privados auxiliares para inicialización K-Means++, asignación y actualización.

DistanceCalculator: Clase que calcula distancias entre respuestas y centroides con múltiples métricas.

Atributos:

- `distanceType`: `DistanceType` privado que indica la métrica a usar (EUCLIDEAN, MANHATTAN).
- `weights`: `Map<Integer, Double>` privado que almacena pesos por pregunta.

Métodos:

- `DistanceCalculator(DistanceType distanceType)`: constructor que establece el tipo de distancia.
- `void setWeight(int questionIndex, double weight)`: establece el peso de una pregunta.
- `Map<Integer, Double> getWeights()`: retorna todos los pesos configurados.
- `DistanceType getDistanceType()`: retorna el tipo de distancia.
- `double calculate(Response rs1, Response rs2, List<Question> questions)`: calcula distancia entre dos respuestas.
- `double calculateToCentroid(Response rs, Centroid centroid, List<Question> questions)`: calcula distancia entre respuesta y centroide.
- `double calculate(Centroid c1, Centroid c2, List<Question> questions)`: calcula distancia entre dos centroides.
- `double euclideanDistance(double[] v1, double[] v2)`: calcula distancia euclídea entre vectores.
- `double calculateTextDistance(String text1, String text2)`: calcula distancia entre textos usando Levenshtein normalizado.
- `double calculateVectorDistance(Object[] vector1, Object[] vector2)`: calcula distancia entre vectores de objetos numéricos.
- `double calculateVectorDistance(Object[] vector1, Object[] vector2, DistanceType type)`: calcula distancia con tipo específico.
- Además de métodos privados para cálculos locales de distancia, Jaccard, Levenshtein, etc.

DistanceType: Enumeración que define los tipos de distancia disponibles.

Valores:

- **EUCLIDEAN**: distancia euclídea (raíz cuadrada de la suma de diferencias al cuadrado).
- **MANHATTAN**: distancia de Manhattan (suma de valores absolutos de diferencias).

ClusteringAnalysis: Clase que representa un análisis de clustering completo sobre una encuesta.

Atributos:

- **id:** String final privado con el ID único del análisis (UUID).
- **surveyId:** String final privado con el ID de la encuesta analizada.
- **k:** Integer final privado con el número de clústeres.
- **algorithmType:** String final privado con el nombre del algoritmo usado.
- **analysisDate:** LocalDateTime final privado con la fecha del análisis.
- **converged:** Boolean privado que indica si convergió.
- **iterations:** Integer privado con el número de iteraciones realizadas.
- **executionTime:** Long privado con el tiempo de ejecución en milisegundos.
- **clusters:** List<Cluster> privada con los clústeres resultantes.
- **algorithm:** ClusteringAlgorithm final privado con el algoritmo usado.
- **responses:** List<Response> privada con las respuestas analizadas.
- **questions:** List<Question> privada con las preguntas de la encuesta.
- **distance:** DistanceCalculator privado usado en el análisis.

Métodos:

- **ClusteringAnalysis(Survey survey, Integer k, ClusteringAlgorithm algorithm):** constructor que inicializa el análisis.
- **String getId():** retorna el ID del análisis.
- **String getSurveyId():** retorna el ID de la encuesta.
- **Integer getK():** retorna el número de clústeres.
- **LocalDateTime getAnalysisDate():** retorna la fecha del análisis.
- **void execute(List<Response> responses, List<Question> questions, DistanceCalculator distance):** ejecuta el análisis de clustering.
- **List<Cluster> getClusters():** retorna los clústeres resultantes.
- **Cluster getCluster(String clusterId):** retorna un clúster por ID.
- **Boolean hasConverged():** retorna si convergió.
- **Integer getIterations():** retorna el número de iteraciones.
- **Long getExecutionTime():** retorna el tiempo de ejecución.
- **List<Response> getResponses():** retorna las respuestas analizadas.
- **List<Question> getQuestions():** retorna las preguntas.
- **Double calculateQuality(QualityMetricType metricType):** calcula una métrica de calidad del clustering.
- **String exportResults():** exporta los resultados a formato de texto.

AnalysisController: Clase controladora que gestiona análisis de clustering.

Atributos:

- **analyses:** Map<String, ClusteringAnalysis> privado que almacena análisis por ID.

Métodos:

- **AnalysisController():** constructor que inicializa el mapa vacío.
- **ClusteringAnalysis createAnalysis(Survey survey, int k, String algorithmName, Map<String, Object> config):** crea un nuevo análisis.
- **void executeAnalysis(String analysisId, List<Response> responses, List<Question> questions, DistanceCalculator distance):** ejecuta un análisis existente.
- **ClusteringAnalysis getAnalysis(String analysisId):** retorna un análisis por ID.

- `void deleteAnalysis(String analysisId)`: elimina un análisis.
- `List<String> getAvailableAlgorithms()`: retorna lista de algoritmos disponibles.
- `void clearAnalyses()`: elimina todos los análisis.
- `Map<String, ClusteringAnalysis> getAnalyses()`: retorna todos los análisis.

1.3.5. Paquete: domain.controller

SurveyController: Clase controladora que gestiona operaciones sobre encuestas, delegando en SurveyService.

Atributos:

- `surveyService`: `SurveyService` final privado que contiene la lógica de negocio.

Métodos:

- `SurveyController(SurveyService surveyService)`: constructor que inyecta el servicio.
- `Survey createSurvey(Survey survey)`: crea una nueva encuesta.
- `Survey initializeNewSurvey(String title, String description, String creatorUsername)`: inicializa una encuesta nueva.
- `List<Survey> getSelectedSurveys()`: retorna encuestas seleccionadas.
- `List<String> getSurveyIds()`: retorna IDs de todas las encuestas.
- `Survey getSurvey(String surveyId)`: retorna una encuesta por ID.
- `List<Survey> getMySurveys()`: retorna encuestas del usuario logueado.
- `boolean existsSurvey(String surveyId)`: verifica si existe una encuesta.
- `void incrementResponseCount(String surveyId)`: incrementa el contador de respuestas.

QuestionController: Clase controladora que gestiona operaciones sobre preguntas, delegando en QuestionService.

Atributos:

- `questionService`: `QuestionService` final privado.

Métodos:

- `QuestionController(QuestionService questionService)`: constructor con inyección de servicio.
- `Question createTextualQuestion(int questionIndex, String surveyId, String questionText, boolean isRequired)`: crea pregunta textual.
- `Question createNumericalQuestion(int questionIndex, String surveyId, String questionText, boolean isRequired)`: crea pregunta numérica.
- `MultipleChoiceQuestion createMultipleChoiceQuestion(int questionIndex, String surveyId, String questionText, boolean isRequired, int minSelections, int maxSelections, List<String> optionTexts)`: crea pregunta de opción múltiple.
- `void updateQuestionText(Question question, String newText)`: actualiza texto de pregunta.
- `void toggleRequiredStatus(Question question)`: cambia el estado de obligatoriedad.
- `void addOption(MultipleChoiceQuestion question, String optionText)`: añade opción.
- `void updateOption(MultipleChoiceQuestion question, int index, String newText)`: actualiza opción.
- `void removeOption(MultipleChoiceQuestion question, int index)`: elimina opción.

- `void updateMinSelections(MultipleChoiceQuestion question, int minSelections):` actualiza mínimo de selecciones.
- `void updateMaxSelections(MultipleChoiceQuestion question, int maxSelections):` actualiza máximo de selecciones.

ResponseController: Clase controladora que gestiona operaciones sobre respuestas, delegando en ResponseService.

Atributos:

`responseService:` ResponseService final privado.

Métodos:

- `ResponseController(ResponseService responseService):` constructor con inyección de servicio.
- `String startResponse(String surveyId):` inicia una nueva respuesta y retorna su ID.
- `List<Question> getQuestions(String surveyId):` retorna preguntas de una encuesta.
- `Question getQuestion(String surveyId, int questionIndex):` retorna una pregunta específica.
- `List<Answer> getAnswers(String surveyId, String responsId):` retorna respuestas de una Response.
- `Question startAnswer(String surveyId, String responsId, int questionIndex):` inicia la respuesta a una pregunta.
- `void updateAnswer(String surveyId, String responsId, int questionIndex, String strAnswer, TypeQuestion typeAnswer):` actualiza respuesta textual o de opción múltiple.
- `void updateAnswer(String surveyId, String responsId, int questionIndex, Double numericalAnswer):` actualiza respuesta numérica.
- `void incrementResponseCount(String surveyId):` incrementa contador de respuestas de la encuesta.

UserController: Clase controladora que gestiona autenticación y operaciones de usuarios.

Atributos:

- `userService:` UserService final privado.
- `usernameLoggedIn:` String privado con el nombre del usuario actual.
- `loggedIn:` boolean privado que indica si hay sesión activa.

Métodos:

- `UserController(UserService userService):` constructor con inyección de servicio.
- `String getUsernameLoggedIn():` retorna el nombre del usuario logueado.
- `boolean isLoggedIn():` retorna si hay sesión activa.
- `void registerUser(String username, String email, String password):` registra un nuevo usuario.
- `void loginUser(String username, String password):` inicia sesión de usuario.
- `void logoutUser():` cierra sesión del usuario.
- `User getLoggedUser():` retorna el objeto User del usuario logueado.
- `void addResponsId(String username, String surveyId, String responsId):` añade ID de respuesta al usuario.
- `void deleteUser(String username):` elimina un usuario.

- `void updateUserEmail(String username, String oldEmail, String newEmail)`: actualiza email del usuario.
- `void changePassword(String username, String email, String password, String newPassword)`: cambia contraseña del usuario.

AnswerController: Clase controladora para operaciones sobre respuestas individuales.

Atributos:

- `answerService: AnswerService` final privado.

Métodos:

- `AnswerController(AnswerService answerService)`: constructor con inyección de servicio.

CtrlDominioClustering: Clase controladora de dominio para operaciones de clustering.

Atributos:

- `analysisController: AnalysisController` final privado.
- `responseRepository: ResponseRepository` final privado.
- `surveyRepository: SurveyRepository` final privado.

Métodos:

- `CtrlDominioClustering(ResponseRepository responseRepository, SurveyRepository surveyRepository)`: constructor que inicializa el controlador.
- `String runAnalysis(String algorithmName, String surveyId, int k, int maxIter, double tolerance, String distanceMetric)`: ejecuta un análisis de clustering y retorna su ID.
- `String exportarAnálisis(String analysisId, String filePath)`: exporta resultados a archivo y retorna la ruta.
- `String obtenerMetricasCalidad(String analysisId)`: obtiene métricas de calidad formateadas.
- `String encontrarKOptima(String surveyId, String algorithmName, int maxIter, double tolerance, String distanceMetric)`: encuentra el valor óptimo de k probando diferentes valores.
- `List<String> listarAnálisis()`: lista todos los IDs de análisis ordenados por fecha.
- `void deleteAnalysis(String analysisId)`: elimina un análisis específico.
- `String obtenerResultadosAnálisis(String analysisId)`: obtiene resultados completos en formato texto.
- `void limpiarAnálisis()`: elimina todos los análisis almacenados.

1.3.6. Paquete: domain.service

SurveyService: Clase de servicio que contiene la lógica de negocio para encuestas.

Atributos:

- `surveyRepository: SurveyRepository` final privado.
- `userController: UserController` final privado.

Métodos:

- `SurveyService(SurveyRepository surveyRepository, UserController userController)`: constructor con inyección de dependencias.

- `Survey createSurvey(Survey survey)`: crea y almacena una nueva encuesta.
- `Survey initializeNewSurvey(String title, String description, String creatorUsername)`: inicializa una encuesta con datos básicos.
- `List<Survey> getSelectedSurveys()`: retorna encuestas disponibles.
- `List<String> getSurveyIds()`: retorna todos los IDs de encuestas.
- `List<Question> getQuestions(String surveyId)`: retorna preguntas de una encuesta.
- `Question getQuestion(String surveyId, int questionIndex)`: retorna una pregunta específica.
- `Survey getSurvey(String surveyId)`: retorna una encuesta por ID.
- `List<Survey> getMySurveys()`: retorna encuestas del usuario logueado.
- `boolean existsQuestion(String surveyId, int questionIndex)`: verifica existencia de pregunta.
- `boolean existsSurvey(String surveyId)`: verifica existencia de encuesta.
- `void incrementResponseCount(String surveyId)`: incrementa contador de visualizaciones.
- Además de métodos privados de validación (`checkUserLoggedIn`, `checkSurveyExists`, `checkQuestionExists`).

QuestionService: Clase de servicio que contiene la lógica de negocio para preguntas.

Atributos:

- `questionRepository`: `QuestionRepository` final privado.
- `userController`: `UserController` final privado.

Métodos:

- `QuestionService(QuestionRepository questionRepository, UserController userController)`: constructor con inyección de dependencias.
- `Question createTextualQuestion(int questionIndex, String surveyId, String questionText, boolean isRequired)`: crea pregunta textual con validaciones.
- `Question createNumericalQuestion(int questionIndex, String surveyId, String questionText, boolean isRequired)`: crea pregunta numérica con validaciones.
- `MultipleChoiceQuestion createMultipleChoiceQuestion(int questionIndex, String surveyId, String questionText, boolean isRequired, int minSelections, int maxSelections, List<String> optionTexts)`: crea pregunta de opción múltiple con validaciones.
- `void updateQuestionText(Question question, String newText)`: actualiza texto con validación.
- `void toggleRequiredStatus(Question question)`: cambia estado de obligatoriedad.
- `void addOption(MultipleChoiceQuestion question, String optionText)`: añade opción con validaciones.
- `void updateOption(MultipleChoiceQuestion question, int index, String newText)`: actualiza opción con validaciones.
- `void removeOption(MultipleChoiceQuestion question, int index)`: elimina opción con validaciones.
- `void updateMinSelections(MultipleChoiceQuestion question, int minSelections)`: actualiza mínimo con validaciones.
- `void updateMaxSelections(MultipleChoiceQuestion question, int maxSelections)`: actualiza máximo con validaciones.

ResponseService: Clase de servicio que contiene la lógica de negocio para respuestas.

Atributos:

- `responseRepository`: `ResponseRepository` final privado.
- `userController`: `UserController` final privado.
- `surveyService`: `SurveyService` público final.

Métodos:

- `ResponseService(ResponseRepository responseRepository, UserController userController, SurveyService surveyService)`: constructor con inyección de dependencias.
- `String startResponse(String surveyId)`: inicia una nueva respuesta y retorna su ID generado.
- `List<Question> getQuestions(String surveyId)`: retorna preguntas de la encuesta.
- `Question getQuestion(String surveyId, int questionIndex)`: retorna pregunta específica.
- `List<Answer> getAnswers(String surveyId, String responsId)`: retorna respuestas con validación de que al menos una está contestada.
- `Question startAnswer(String surveyId, String responsId, int questionIndex)`: inicia respuesta a pregunta específica.
- `void updateAnswer(String surveyId, String responsId, int questionIndex, String strAnswer, TypeQuestion answerType)`: actualiza respuesta textual o de opción múltiple con validaciones.
- `void updateAnswer(String surveyId, String responsId, int questionIndex, Double numAnswer)`: actualiza respuesta numérica con validaciones.
- `void incrementResponseCount(String surveyId)`: incrementa contador de respuestas.
- `boolean[] getOptionsSelected(String input, int minSelections, int maxSelections, int numOptions)`: parsea y valida opciones seleccionadas.
- Además de métodos privados de validación y utilidad.

UserService: Clase de servicio que contiene la lógica de negocio para usuarios.

Atributos:

- `userRepository`: `UserRepository` final privado.

Métodos:

- `UserService(UserRepository userRepository)`: constructor con inyección del repositorio.
- `void registerUser(String username, String email, String password)`: registra usuario con validaciones (username único, campos no vacíos).
- `void verifyCredentials(String username, String password)`: verifica credenciales para login.
- `User getUser(String username)`: retorna usuario por nombre.
- `boolean deleteUser(String username)`: elimina usuario y retorna éxito.
- `String updateUserEmail(String username, String oldEmail, String newEmail)`: actualiza email con validaciones y retorna código de resultado.
- `String changePassword(String username, String email, String password, String newPassword)`: cambia contraseña con validaciones y retorna código de resultado.
- `void addResponsId(String username, String surveyId, String responsId)`: añade ID de respuesta al usuario.

- Además de métodos privados de validación.

AnswerService: Clase de servicio para operaciones sobre respuestas individuales.

Atributos:

- **answerRepository:** `AnswerRepository` final privado.
- **userController:** `UserController` final privado.

Métodos:

- **AnswerService(AnswerRepository answerRepository, UserController userController):** constructor con inyección de dependencias.

1.3.7. Paquete: domain.exception

- **SurveyException:** Clase de excepción para errores relacionados con encuestas, extiende `RuntimeException`.
 - **Métodos:** `SurveyException(String message)`: constructor que acepta mensaje de error.
- **ResponseException:** Clase de excepción para errores relacionados con respuestas, extiende `RuntimeException`.
 - **Métodos:** `ResponseException(String message)`: constructor que acepta mensaje de error.
- **LogInException:** Clase de excepción para errores de autenticación, extiende `RuntimeException`.
 - **Métodos:** `LogInException(String message)`: constructor que acepta mensaje de error.
- **RegisterException:** Clase de excepción para errores de registro, extiende `RuntimeException`.
 - **Métodos:** `RegisterException(String message)`: constructor que acepta mensaje de error.